

Módulo 11 – Capítulo 06 – Sección 01

RAG empresarial: desafíos de seguridad, autorización y segmentación de conocimiento

El RAG (Retrieval-Augmented Generation) en contexto enterprise introduce un mecanismo de acceso a información que los sistemas de búsqueda convencionales no tienen: la capacidad del LLM de sintetizar múltiples fragmentos de documentos en una respuesta coherente. Esta capacidad de síntesis es precisamente lo que hace que el RAG empresarial sea valioso — y precisamente lo que lo convierte en un vector de riesgo de seguridad que no existe en los motores de búsqueda tradicionales. Cuando un motor de búsqueda muestra un documento, el usuario ve el documento y puede juzgar si tiene derecho a consultarlo. Cuando un sistema RAG sintetiza una respuesta de veinte fragmentos de documentos de distintos niveles de clasificación, el usuario recibe una respuesta sin saber de qué documentos proviene, y el sistema es el responsable de garantizar que esa respuesta solo contiene información que el usuario está autorizado a ver.

El problema de la **segmentación de conocimiento** emerge cuando el sistema RAG indexa documentos de diferentes niveles de clasificación — público, interno, confidencial, restringido — en el mismo índice vectorial sin filtros de autorización en la capa de retrieval. Una búsqueda semántica sin filtros puede recuperar fragmentos de documentos restringidos en el contexto de una petición de un usuario que solo tiene acceso a documentos internos. El LLM, que no tiene visibilidad de los permisos del usuario, generará una respuesta que sintetiza toda la información disponible en el contexto — incluyendo la información del documento restringido — produciendo una exfiltración de datos que ningún sistema de monitoreo de red convencional detectaría, porque la información viajó en el payload de una petición HTTP perfectamente legítima.

La **temporalidad del acceso** es la dimensión más técnicamente desafiante del control de acceso en RAG enterprise. Cuando un usuario pierde el acceso a un documento porque cambió de rol, porque el documento fue reclasificado, o porque venció su autorización temporal, el índice vectorial debe reflejar ese cambio de acceso de manera inmediata o con una latencia máxima documentada y aprobada por el CISO. A diferencia de una base de datos

relacional donde el control de acceso se implementa en la query y es inmediatamente efectivo, el índice vectorial almacena los embeddings de los documentos de manera independiente de los permisos de acceso, y el control de acceso debe implementarse como filtro en la query vectorial, consultando en tiempo real (o con caché de TTL corto) el sistema de IAM del enterprise.

La segmentación de conocimiento por dominio — cada departamento ve solo los documentos de su dominio, con excepciones gestionadas por el sistema de gestión de permisos del DMS — es la dimensión más crítica para el cumplimiento en sectores regulados. La conexión con el Capítulo 04 sobre multi-tenancy es directa: el aislamiento de índices vectoriales por tenant descrito en la Sección 02 de ese capítulo es exactamente el mecanismo que garantiza la segmentación de conocimiento en sistemas RAG multi-tenant — los documentos del departamento legal de un tenant nunca aparecen en los resultados de retrieval de una petición del departamento financiero de ese mismo tenant, porque están en colecciones distintas con permisos distintos.

Desafíos técnicos del RAG empresarial

- **Permission-aware retrieval:** el pipeline de recuperación debe filtrar los documentos candidatos por los permisos del usuario autenticado antes de ejecutar la búsqueda vectorial de similitud, no después — el post-filtering es inútil porque el daño (el LLM procesó información no autorizada) ya se produjo.
- **Clasificación de documentos en el índice:** cada chunk vectorizado lleva metadata de su nivel de clasificación (public, internal, confidential, restricted) y los permisos del documento fuente (lista de grupos autorizados), usada como filtro en cada query vectorial.
- **Acumulación de contexto y information leakage:** el LLM recibe múltiples chunks de documentos distintos en el mismo contexto; si alguno proviene de un documento no autorizado para el usuario, hay un fallo de control de acceso que el LLM no puede detectar ni corregir.
- **Revocación de acceso en tiempo real:** implementada mediante filtros dinámicos basados en permisos actualizados con caché de TTL corto (5-60 segundos) — el TTL determina el tiempo máximo de exposición residual tras la revocación.
- **Audit logging de retrieval:** registro de cada búsqueda vectorial con el user_id del solicitante, los document IDs recuperados, y los chunks incluidos en el contexto del LLM, para auditoría forense de acceso a información sensible.

Para recordar: Un sistema RAG enterprise que no implementa autorización a nivel de documento en la capa de retrieval es un sistema que potencialmente exfiltra información confidencial a cualquier usuario autenticado que formule la pregunta semánticamente correcta. La autenticación del usuario no sustituye la autorización a nivel de documento.

La sección siguiente profundiza en el mecanismo técnico específico del permission-aware retrieval: cómo se implementa el pre-filtering en Pinecone, Weaviate, y pgvector, y cuáles son sus trade-offs de rendimiento y seguridad.

Módulo 11 – Capítulo 06 – Sección 02

Document-level permissions: que el sistema RAG respete el control de acceso de los documentos fuente

La implementación técnica del control de acceso en sistemas RAG enterprise tiene una propiedad contraintuitiva que sorprende a los equipos que la encuentran por primera vez: filtrar los resultados en la capa de presentación, después de que el LLM ha generado su respuesta, no proporciona ninguna protección de seguridad real. Si el LLM recibió en su contexto un fragmento de un documento clasificado como "confidencial" antes de generar la respuesta, esa información ya fue procesada y puede estar presente en la respuesta generada, aunque el sistema filtre la respuesta antes de mostrarla al usuario. El control de acceso real en sistemas RAG debe implementarse en la **capa de retrieval**, antes de que ningún documento no autorizado entre en el contexto del LLM.

El mecanismo técnico del **pre-filtering** funciona así: antes de ejecutar la búsqueda por similitud vectorial, el sistema resuelve los grupos de permisos del usuario autenticado consultando al Identity Provider (Okta, Azure AD, o el LDAP corporativo) y construye un filtro de metadata que restrinja la búsqueda vectorial a los documentos donde el usuario tiene permiso de lectura. En Pinecone, el filtro se especifica como un parámetro `filter` en la query vectorial con operadores `$in` (el campo `allowed_groups` del documento contiene al menos uno de los grupos del usuario) — la búsqueda vectorial opera solo sobre los vectores que pasan el filtro de metadata, no sobre el índice completo. En Weaviate, los `where` filters cumplen la misma función. En pgvector sobre PostgreSQL, las políticas de Row Level Security implementan el filtro de manera automática e incluso más robusta: el filtro se aplica a nivel del motor de base de datos, independientemente de cómo la aplicación construya la query SQL.

El desafío de rendimiento de este enfoque es real y no debe ignorarse. La lista de grupos de permisos del usuario puede ser larga — un ejecutivo senior puede pertenecer a 50-100 grupos de seguridad — y los filtros de metadata en bases de datos vectoriales tienen un overhead de rendimiento proporcional a la complejidad del filtro. La solución estándar es un **permissions-resolver service** que cachea en Redis el resultado de la consulta al IAM por

`user_id` con un TTL de 5-10 minutos, reduciendo el número de consultas al IAM corporativo de una por petición a una cada 5-10 minutos por usuario activo. Esta caché introduce una latencia de revocación — en el peor caso, un usuario al que se le revocó el acceso puede seguir accediendo a documentos durante el TTL de la caché — que debe ser explícitamente documentada y aprobada por el CISO como parte del contrato de seguridad del sistema RAG.

La **herencia de permisos** añade una capa adicional de complejidad: cuando un directorio en SharePoint tiene permisos asignados, todos los documentos dentro de ese directorio heredan esos permisos a menos que tengan permisos explícitos propios. Esta lógica de herencia, que los sistemas DMS como SharePoint gestionan automáticamente, debe replicarse exactamente en el pipeline de indexación del RAG: cuando se indexa un documento, el sistema debe resolver los permisos efectivos del documento incluyendo los heredados del directorio, y almacenarlos como metadata junto al vector. Sin esta replicación, el índice vectorial puede tener permisos incorrectos que no reflejan el estado real del DMS.

Aspectos técnicos de document-level permissions

- **Permission metadata en embeddings:** cada documento se vectoriza con metadata que incluye `document_acl` (lista de grupos con acceso de lectura), `classification_level`, y `owner_department`, almacenada como campos filtrables en el índice vectorial.
- **Pre-filtering vs post-filtering:** pre-filtering (filtro antes de la búsqueda vectorial) garantiza que ningún documento no autorizado entra en el contexto del LLM; post-filtering no proporciona protección real porque el LLM ya procesó la información.
- **Permissions resolver con caché:** servicio que consulta Azure AD Graph API o LDAP, cachea el resultado por `user_id` en Redis con TTL de 5-10 minutos, y expone el permission set como lista de `group_ids` para construir el filtro vectorial.
- **Herencia de permisos:** el pipeline de indexación debe resolver los permisos efectivos de cada documento incluyendo los heredados del directorio contenedor en el DMS, actualizando la metadata del vector cuando los permisos del directorio cambian.
- **Revocación y TTL de caché:** el contrato de seguridad del sistema RAG debe documentar explícitamente el tiempo máximo de revocación de acceso (equivalente al TTL del caché de permisos), con aprobación explícita del CISO como parte del proceso de diseño de seguridad.

Buena práctica: Documentar explícitamente en el security design document del sistema RAG el tiempo máximo de revocación de acceso — el tiempo que puede transcurrir entre que el IAM revoca un permiso y que el sistema RAG refleja ese cambio. Este parámetro (equivalente al TTL del caché de permisos) debe ser aprobado formalmente por el CISO, no decidido unilateralmente por el equipo de ingeniería.

Con el control de acceso a nivel de documento establecido, la sección siguiente amplía el scope del RAG enterprise más allá de los documentos no estructurados para incluir los datos estructurados que residen en bases de datos y data warehouses — el RAG híbrido que cubre el espectro completo de necesidades de información de un enterprise.

Módulo 11 – Capítulo 06 – Sección 03

RAG sobre datos estructurados y no estructurados en el enterprise

Los datos enterprise se distribuyen entre dos categorías con propiedades de retrieval fundamentalmente distintas, y la mayoría de las preguntas de negocio relevantes requieren acceso a ambas. Los **datos no estructurados** — documentos PDF, presentaciones PowerPoint, correos electrónicos, tickets de soporte, transcripciones de reuniones, contratos — se indexan como texto mediante chunking y embedding para búsqueda vectorial: la pregunta "¿cuáles son los términos de rescisión del contrato con el proveedor X?" requiere recuperar el fragmento específico del contrato que contiene esa cláusula. Los **datos estructurados** — tablas de bases de datos relacionales, data warehouses, APIs de sistemas ERP, hojas de cálculo — se consultan mejor mediante queries SQL precisas: la pregunta "¿cuánto hemos facturado al cliente X en el último trimestre?" requiere una agregación exacta sobre la tabla de transacciones, no un retrieval semántico sobre embeddings de los registros de facturación.

El problema del RAG exclusivamente vectorial sobre datos estructurados es evidente: generar embeddings de filas de base de datos para buscarlas por similitud semántica es técnicamente posible pero conceptualmente inadecuado para la mayoría de las consultas financieras o transaccionales, donde la exactitud matemática importa más que la similitud semántica. Si el usuario pregunta "¿cuál es la facturación del Q3?", una búsqueda vectorial recuperará filas que son semánticamente similares a la pregunta, no la suma exacta de las transacciones del tercer trimestre.

El patrón **Text-to-SQL** resuelve este problema para datos estructurados: el LLM convierte la pregunta en lenguaje natural en una query SQL ejecutable sobre la base de datos. La ventaja es que los datos están siempre actualizados (la query se ejecuta contra el estado actual de la base de datos, no contra un índice vectorial que puede estar desactualizado) y la precisión es matemática. El riesgo es que el LLM puede generar SQL incorrecto, SQL que accede a tablas no autorizadas, o SQL computacionalmente costoso que satura el servidor de base de datos. Estos riesgos se mitigan con guardrails específicos: validación de la query SQL con EXPLAIN

antes de ejecutarla, whitelist de tablas y columnas accesibles para el LLM (schema masking: el LLM solo ve el schema relevante para su dominio), y timeout de 5 segundos que previene queries de larga duración.

El **RAG híbrido** que combina retrieval vectorial con Text-to-SQL es la arquitectura que cubre el espectro completo de necesidades de información enterprise. Un router — implementado como un clasificador de intención de la pregunta, que puede ser tan simple como un LLM con un prompt de routing — analiza la pregunta del usuario y determina la estrategia de retrieval apropiada: retrieval vectorial para preguntas sobre política interna, contratos, y documentación técnica; Text-to-SQL para preguntas sobre métricas, cifras, y datos transaccionales; o una combinación de ambos para preguntas que requieren contexto documental y datos cuantitativos simultáneamente (por ejemplo: "¿el nivel de servicio del proveedor X cumple con el SLA acordado en el contrato?" requiere recuperar el SLA del contrato mediante RAG vectorial y los datos de cumplimiento del servicio mediante Text-to-SQL).

Nota del Arquitecto: El schema masking — proporcionar al LLM solo el subset del schema de base de datos relevante para su dominio — es tanto un control de seguridad como una optimización de calidad. Sin schema masking, el LLM puede generar queries sobre tablas que el usuario no tiene autorización de consultar; con un schema completo de decenas de tablas, el LLM produce queries menos precisas porque la información irrelevante en el prompt interfiere con la tarea. Mantener catálogos de schema por dominio de usuario que se sirven dinámicamente al LLM según el contexto es la implementación correcta.

Componentes del RAG sobre datos heterogéneos

- **Chunking semántico para no estructurados:** estrategia de chunking con LangChain SemanticChunker o LlamaIndex SentenceWindowNodeParser que produce chunks de 512-1024 tokens con overlap del 20%, preservando el contexto entre chunks adyacentes y respetando los límites naturales del texto.
- **Text-to-SQL con schema masking:** el LLM recibe solo el schema relevante para el dominio del usuario, no el schema completo de la base de datos, reduciendo el riesgo de queries sobre tablas no autorizadas y mejorando la precisión de las queries generadas.
- **Validación de SQL generado:** ejecución en modo EXPLAIN antes de la ejecución real para verificar el plan de query, comprobación de que la query solo accede a tablas en el

whitelist configurado, y timeout de 5 segundos para prevenir queries costosas.

- **Reranker cross-encoder:** modelo de reranking (ms-marco-MiniLM-L-12-v2, Cohere Rerank, Jina Rerank) que reordena los chunks recuperados por similitud vectorial usando un score de relevancia contextual más preciso antes de incluirlos en el contexto del LLM.
 - **Router de estrategia de retrieval:** clasificador de intención que determina si la pregunta requiere RAG vectorial, Text-to-SQL, o una combinación de ambos, basándose en palabras clave, estructura de la pregunta, y el historial del contexto conversacional.
-

Idea central: El RAG híbrido que combina retrieval vectorial para datos no estructurados con Text-to-SQL para datos estructurados es la arquitectura que cubre el espectro completo de necesidades de información de un enterprise, donde la mayoría de las preguntas de negocio estratégicas requieren ambos tipos de contexto para ser respondidas con precisión.

La sección siguiente aborda la dimensión temporal del RAG enterprise: cómo mantener el índice vectorial actualizado con el flujo continuo de nuevos documentos corporativos sin sacrificar la disponibilidad del sistema ni incurrir en costos de reindexación completa.

Módulo 11 – Capítulo 06 – Sección 04

Knowledge management: actualización continua del índice con nuevos documentos corporativos

Un índice vectorial construido una vez y nunca actualizado es un sistema que envejece activamente: cada día que pasa sin reindexación, el índice diverge del estado actual del conocimiento corporativo. Las políticas que fueron actualizadas, los contratos que fueron firmados o cancelados, los procedimientos que fueron revisados — todos producen nuevos documentos que no están disponibles para el sistema RAG hasta que se reindexan, y documentos antiguos que siguen presentes en el índice aunque ya no sean la versión vigente. En un sistema de soporte interno que responde preguntas sobre la política de permisos de vacaciones, tener en el índice simultáneamente la versión de 2023 y la de 2024 — con reglas que podrían contradecirse — produce respuestas que mezclan información de ambas versiones, confundiendo al usuario y generando errores de proceso que tienen consecuencias reales.

El **pipeline de actualización incremental** del índice vectorial se activa mediante eventos del sistema de gestión documental. Cuando SharePoint Online, Confluence Cloud, o el DMS corporativo notifica un cambio mediante webhook (evento de tipo DocumentCreated, DocumentModified, o DocumentDeleted), el pipeline identifica el document_id afectado, elimina todos los chunks del índice vectorial con ese document_id, rechunkea y re-embeddea el documento actualizado, e inserta los nuevos chunks con el timestamp de la versión actual. Este proceso — delete-and-reindex en lugar de update-in-place — es el más simple de implementar correctamente: los embeddings son generados a partir del contenido y no pueden actualizarse parcialmente sin generar incoherencias.

La arquitectura event-driven de este pipeline se construye sobre el mismo backbone de Kafka descrito en el Capítulo 02: los eventos del DMS se publican en un topic de Kafka, un consumer group de workers de indexación los procesa, y el índice vectorial refleja el estado actual del DMS con una latencia determinada por la velocidad del pipeline. El monitoring de esta latencia — el tiempo entre la modificación del documento en el DMS y su disponibilidad actualizada en el índice vectorial — es la métrica de freshness del sistema RAG, y debe tener

un SLO definido (por ejemplo, menos de 5 minutos para documentos de política crítica, menos de 24 horas para documentos de referencia).

La **eliminación de documentos** del índice vectorial es el caso más crítico y el más frecuentemente ignorado en las implementaciones iniciales. Cuando un documento se revoca — una política obsoleta que ya no debe estar disponible para el sistema RAG — sus embeddings deben eliminarse del índice para que no sigan siendo recuperados en búsquedas futuras. En Pinecone, la eliminación se hace mediante el ID del vector; en Weaviate, mediante la UUID del objeto; en pgvector, mediante DELETE con el `document_id`. El mecanismo de eliminación requiere que el pipeline mantenga un mapa de `document_id` → `chunk_ids`: sin este mapa, no es posible identificar qué vectores deben eliminarse cuando se revoca un documento.

El **freshness scoring** — la penalización de los chunks de documentos muy antiguos cuando existen versiones más recientes disponibles — es una optimización más sofisticada que solo está justificada para casos de uso donde la antigüedad del documento es un indicador de su relevancia. En documentos de política interna, un documento de hace 5 años puede seguir siendo completamente vigente si no ha sido actualizado. En documentos de precios o condiciones de servicio, una versión de hace 6 meses puede estar completamente desactualizada. El freshness scoring debe configurarse específicamente para cada tipo de documento, no aplicarse uniformemente al corpus completo.

Aspectos técnicos del knowledge management

- **Event-driven indexing:** webhooks de SharePoint Online, Confluence Cloud, o S3 Event Notifications que disparan el pipeline de indexación incremental mediante eventos en Kafka, procesados por workers de indexación con dead letter queues para manejo de fallos.
- **Document versioning en el índice:** cada chunk almacena el `document_id`, el hash SHA-256 del contenido (para detectar si el documento cambió realmente o solo sus metadatos), y el timestamp de indexación.
- **Delete-and-reindex:** cuando un documento se modifica o elimina, todos sus chunks se eliminan del índice y se reinsertan los chunks del documento actualizado — nunca update-in-place, que produce incoherencias entre el vector y el contenido del chunk.
- **Mapa `document_id` → `chunk_ids`:** tabla auxiliar en PostgreSQL que mantiene el listado de vector IDs generados por cada documento, necesaria para poder eliminar todos los chunks de un documento cuando se revoca o se actualiza.

- **Monitoring de lag de indexación:** alerta cuando el tiempo entre la modificación de un documento en el DMS y su disponibilidad actualizada en el índice vectorial supera el SLO definido, con granularidad por tipo de documento y por prioridad de negocio.
-

Buena práctica: Implementar un test de regresión de knowledge currency que verifique semanalmente que los documentos más críticos del corpus — las 50 políticas más consultadas, los contratos activos más importantes — tienen su versión actualizada en el índice vectorial. Este test detecta fallas silenciosas en el pipeline de actualización antes de que afecten a los usuarios.

Con el knowledge management establecido como proceso continuo, la sección siguiente aterriza el RAG enterprise en los cuatro casos de uso más frecuentes y de mayor impacto en organizaciones reales, con sus particularidades técnicas específicas.

Módulo 11 – Capítulo 06 – Sección 05

Casos de uso enterprise: soporte interno, asistentes de ventas, análisis de contratos y RAG sobre código

El RAG enterprise no es una solución única: cada caso de uso tiene requisitos técnicos específicos que determinan la arquitectura del pipeline de retrieval, la estrategia de chunking, las métricas de evaluación relevantes, y los controles de calidad necesarios. Entender estas diferencias antes de diseñar el sistema evita la trampa de construir un pipeline genérico que funciona razonablemente para ningún caso de uso en particular, en lugar de uno especializado que funciona excelentemente para el caso de uso que el negocio necesita.

El **asistente de soporte interno** (HR helpdesk, IT support, Legal FAQ) es el caso de uso de menor riesgo y mayor velocidad de despliegue por varias razones estructurales: el corpus es relativamente estable (las políticas de HR y los procedimientos de IT se actualizan con frecuencia mensual o trimestral, no diaria), los usuarios son empleados autenticados con permisos conocidos, el impacto de una respuesta incorrecta es bajo (el empleado puede verificar la política directamente), y las métricas de éxito son claramente cuantificables (reducción del 30-50% en tickets de soporte de primer nivel). La arquitectura técnica es la más simple: chunking semántico del corpus de documentos de política, embeddings con text-embedding-3-large, índice por departamento para respetar la segmentación, y evaluación con golden set de las 50 preguntas más frecuentes del helpdesk histórico.

El **asistente de ventas** (product information, pricing, competitive intelligence) introduce una complejidad crítica: la frescura de los datos. Los precios, la disponibilidad de productos, y las condiciones especiales cambian con alta frecuencia — diaria en algunos sectores como el comercio electrónico o el sector financiero — y la información desactualizada tiene un impacto directo y medible: un vendedor que cita un precio incorrecto pierde la negociación o genera un contrato con condiciones que la empresa no puede cumplir. La arquitectura debe integrar RAG vectorial sobre la documentación de producto (relativamente estable) con Text-to-SQL sobre el sistema PIM o ERP para precios y disponibilidad en tiempo real, con alertas cuando la información del índice tiene más de 24 horas de antigüedad para documentos de precio.

El **análisis de contratos** es el caso de uso con mayor impacto por documento procesado y mayor exigencia de precisión. Los errores de extracción pueden tener consecuencias legales: si el sistema extrae incorrectamente una cláusula de penalización, la empresa puede operar durante meses bajo el supuesto de condiciones contractuales incorrectas. Por esta razón, el sistema de análisis de contratos debe implementarse como **asistente de revisión** (herramienta que asiste al abogado en su análisis) no como tomador de decisiones autónomo, y debe presentar siempre la cita exacta del texto del contrato junto a cada extracción, permitiendo al revisor humano verificar la fuente original. El pipeline incluye procesamiento de PDF con extracción de texto estructurado (Unstructured.io, pdfplumber para PDFs con texto; Azure Document Intelligence para PDFs escaneados), clasificadores de tipo de cláusula, y modelos de extracción con Pydantic models que validan los campos extraídos.

El **RAG sobre codebase** (code assistant con contexto del repositorio corporativo) tiene requisitos de embeddings distintos a los documentos de texto: el código tiene una estructura semántica diferente del lenguaje natural, y los embeddings de texto general (text-embedding-3-large) producen retrieval de menor calidad para código que los embeddings especializados. Herramientas como Voyage Code-3 o CodeBERT están entrenadas específicamente en código fuente y producen mejores resultados de retrieval para preguntas sobre implementaciones, patrones de arquitectura, y uso de librerías internas. El chunking debe respetar los límites sintácticos del código — por función o por clase — no simplemente por número de tokens, para garantizar que cada chunk es una unidad semántica coherente que puede entenderse de manera independiente.

Características técnicas por caso de uso

- **Soporte interno:** corpus de 1.000-10.000 documentos de política y procedimientos, chunking semántico con overlap del 20%, filtros de departamento para segmentación de acceso, latencia de respuesta < 3 segundos, y métrica principal de deflection rate de tickets (objetivo: reducción del 30-50%).
- **Asistente de ventas:** integración con PIM y ERP para datos de precios en tiempo real via Text-to-SQL, corpus de documentación de producto para RAG vectorial, alertas de freshness cuando la información tiene más de 24 horas de antigüedad, y validación de precios contra la fuente authoritative antes de presentarlos.
- **Análisis de contratos:** pipeline de PDF con extracción de texto estructurado, clasificadores de tipo de cláusula (penalización, terminación, renovación automática, SLA), extracción con Pydantic models validados, UI de revisión con citas exactas del texto original, y presentación siempre como herramienta de asistencia al revisor humano.

- **RAG sobre codebase:** indexación de repositorios Git con tree-sitter para parsing sintáctico, chunking por función o clase respetando límites semánticos, embeddings especializados en código (Voyage Code-3), y filtros por repositorio y rama según permisos de acceso al código del usuario.
-

Para recordar: Cada caso de uso de RAG enterprise tiene una métrica de negocio específica que debe definirse antes de la implementación y medirse continuamente en producción. Sin esa métrica, es imposible demostrar el valor del sistema una vez en producción, justificar la inversión adicional en mejoras, o detectar cuándo el sistema deja de generar valor.

La sección de cierre sintetiza el argumento central del capítulo sobre RAG enterprise: que la complejidad de estos sistemas no está en la sofisticación del retrieval sino en la robustez del control de acceso y la frescura del conocimiento indexado.

Módulo 11 – Capítulo 06 – Sección 06

Cierre: el RAG empresarial añade una capa de complejidad que es principalmente de acceso y gobierno

El RAG empresarial convierte el sistema de búsqueda de información de la organización en una interfaz conversacional sobre el conocimiento corporativo, y ese salto de capacidad viene acompañado de una responsabilidad de control de acceso que no existe en los motores de búsqueda convencionales. La distinción es fundamental: cuando un motor de búsqueda muestra un documento en respuesta a una consulta, el usuario ve el documento completo con su clasificación visible, puede juzgar si debería estar leyéndolo, y el sistema no ha procesado ni sintetizado nada. Cuando un sistema RAG sintetiza una respuesta de veinte fragmentos de documentos, el usuario recibe información sin ver los documentos fuente, sin saber su clasificación, y sin poder juzgar si la síntesis mezcla información de distintos niveles de acceso. El sistema RAG es el custodio de ese control, y si falla, el fallo es silencioso y difícil de detectar.

Los patrones cubiertos en este capítulo — el permission-aware retrieval con pre-filtering como único mecanismo de control válido, el document-level permissions con herencia de ACLs del DMS, el RAG híbrido que combina retrieval vectorial con Text-to-SQL para datos estructurados y no estructurados, el knowledge management con indexación event-driven que mantiene la frescura del corpus, y los cuatro casos de uso enterprise con sus particularidades técnicas específicas — constituyen el conjunto de decisiones que separan un sistema RAG experimental de un sistema RAG apto para operar en producción enterprise con datos sensibles.

La complejidad del RAG enterprise no reside en la sofisticación del algoritmo de retrieval — HNSW, FAISS, y las implementaciones de Pinecone y Weaviate son algorítmicamente maduras — sino en la robustez del control de acceso y en la frescura del conocimiento indexado. Un sistema RAG con el mejor reranker del mercado pero sin control de acceso a nivel de documento es un sistema de seguridad que falla. Un sistema RAG con control de acceso robusto pero con un índice que tiene 6 meses de retraso es un sistema que produce respuestas desactualizadas que erosionan la confianza del usuario. Los dos ejes — seguridad y

frescura — deben mantenerse simultáneamente como propiedades de primer nivel del sistema.

El siguiente capítulo cambia el eje de análisis de la calidad técnica a la sostenibilidad económica: cómo gestionar los costos de IA a escala enterprise, dónde se concentra el mayor potencial de optimización, y cómo construir el framework de ROI que justifica la inversión ante el liderazgo con datos verificables de producción.

"Los datos son el activo más valioso de una empresa, pero solo si pueden encontrarse y usarse de manera confiable — y eso requiere tanto ingeniería de datos rigurosa como controles de acceso precisos." — DJ Patil, primer Chief Data Scientist de los Estados Unidos